

Institute of Science & Technology

Department of Computer Science and Engineering

Result Processing System

Role Based Academic Result Management

A project report submitted in partial fulfillment of the requirements
for the degree of Bachelor of Computer Science and Engineering

Submitted By

Kazi Saif Hossain Emon

Registration No 21502005399

Roll No 22075

Session 2021–2022

Supervised By

Saleh Muhammad Maruf

Assistant Professor

Department of Computer Science and Engineering

Date of Submission 4 November 2026

ABSTRACT

Academic result preparation requires consistent course registration, controlled marks entry and a clear decision about when results become visible to students. The Result Processing System (RPS) addresses these needs through a role-based web application that connects academic records, course assignments, assessment marks and result publication in a single local workflow. The objective is to reduce repeated data handling and prevent common errors involving batch identity, unauthorized access and premature result disclosure.

The application is implemented with FastAPI, SQLAlchemy and SQLite, supported by an HTML, CSS and JavaScript interface. Administrators maintain academic data and accounts, assign teachers to session-specific courses, configure batch semesters and publish reviewed results. Teachers enter four validated assessment components within their assigned scope. Students view their own published results, including credit-weighted semester GPA and CGPA. Fixed core-course enrollment, CSV import validation, revision checks and publication locks support data consistency.

Evaluation in this report uses isolated unit tests of grading functions, marks validation and frontend state handlers. Twelve Python test methods and six JavaScript test methods passed. Interface snapshots illustrate the implemented role-specific workspaces using demonstration records. The project provides a practical local academic result-processing prototype. Institution-approved grading policies, retakes, audit history and public deployment remain areas for further development.

Keywords: Result Processing System, academic records, role-based access control, result publication, GPA, CGPA.

DECLARATION

I, Kazi Saif Hossain Emon, Registration No 21502005399, Roll No 22075, Session 2021–2022, of the Department of Computer Science and Engineering at the Institute of Science & Technology, declare that this project report titled “Result Processing System” presents the software project prepared for my academic submission under the supervision of Saleh Muhammad Maruf, Assistant Professor.

The report explains the requirements, design, implementation and unit testing of the project. Technical ideas and documentation used in preparing the report are acknowledged through the references. The project is submitted for academic evaluation; demonstration results shown in the report are not official academic records.

Signature _____

Kazi Saif Hossain Emon
Registration No 21502005399
Roll No 22075
Session 2021–2022

Date 4 November 2026

ACKNOWLEDGEMENT

I express my gratitude to Almighty Allah for the strength and opportunity to complete this project and report.

I sincerely thank my supervisor, Saleh Muhammad Maruf, Assistant Professor in the Department of Computer Science and Engineering at the Institute of Science & Technology, for academic guidance and supervision of this work.

I am grateful to the teachers and staff of the department for the learning environment and support provided during my studies. I also thank my classmates for their encouragement and the project report shared as a reference for the organization of this book.

I acknowledge the developers and maintainers of the open-source technologies and technical documentation used in the Result Processing System. Finally, I thank my parents and family for their patience, encouragement and support throughout my education.

Kazi Saif Hossain Emon

CONTENTS

Abstract.....i

Declaration.....ii

Acknowledgement.....iii

Chapter 1 Introduction.....1

1.1 Definition of the system.....1

1.2 Problem statement.....1

1.3 Purpose.....1

1.4 Report organization.....1

1.5 Objectives.....2

1.6 Scope of the implemented system.....2

1.7 Project boundaries.....2

1.8 Academic batch configuration.....2

Chapter 2 Literature review.....3

2.1 Academic result-processing approaches.....3

2.2 Role-based access control.....3

2.3 Authentication and validation.....3

2.4 Relational integrity and transactions.....4

2.5 Optimistic concurrency.....4

2.6 Scope comparison.....4

Chapter 3 Feasibility study.....5

3.1 Operational feasibility.....5

3.2 Technical feasibility.....5

3.3 Economic feasibility.....5

3.4 Adoption and schedule feasibility.....5

Chapter 4 Requirement analysis.....6

4.1 Development methodology.....6

4.2 Actors and responsibilities.....6

CONTENTS CONTINUED

4.3 Functional requirement groups.....6
4.4 Data privacy requirements.....6
4.5 Core business rules.....8
4.6 Software and hardware environment.....8
Chapter 5 System design.....9
5.1 Architectural design.....9
5.2 Input and output design.....9
5.3 Use case diagram.....10
5.4 Data flow diagram.....11
5.5 Transaction boundaries.....11
5.6 Entity relationships.....12
5.7 Result processing sequence.....13
Chapter 6 Implementation and results.....14
6.1 Application structure.....14
6.2 Database initialization and migration.....14
6.3 Account lifecycle.....14
6.4 Assessment total and grade mapping.....15
6.5 GPA and CGPA.....15
6.6 Login interface and first access.....16
6.7 Interface conventions.....16
6.8 Dashboard and batch summary.....17
6.9 Teacher assignment and marks entry.....18
6.10 Administrative review.....19
6.11 Student result viewing.....20

CONTENTS CONTINUED

6.12 Implemented outcome....20
Chapter 7 Testing....21
7.1 Unit testing approach....21
7.2 Test environment and isolation....21
7.3 Selection of cases....21
7.4 Backend unit cases....22
7.5 Frontend unit cases....22
7.6 Observed execution output....23
7.7 Interpretation and limits....23
Chapter 8 Conclusions and future work....24
8.1 Conclusion....24
8.2 Limitations....24
8.3 Future development....24
8.4 Completion status....24
References....25

LIST OF FIGURES

Figure 5.1 Layered architecture of the Result Processing System.....9
Figure 5.2 Role-specific use cases.....10
Figure 5.3 Main academic data flows and logical data stores.....11
Figure 5.4 Database entities and principal relationships.....12
Figure 5.5 Save review publish and view sequence.....13
Figure 6.1 Redesigned login interface.....16
Figure 6.2 Administrative dashboard with batch progress.....17
Figure 6.3 Teacher marks entry in an assigned session.....18
Figure 6.4 Component-level result review before publication.....19
Figure 6.5 Student result view with verified GPA and CGPA.....20
Figure 7.1 Captured Python and frontend unit test output.....23

LIST OF TABLES

Table 2.1 Comparison of result-processing approaches....4
Table 3.1 Development milestones....5
Table 4.1 Role responsibility matrix....6
Table 4.2 Functional requirements....7
Table 4.3 Nonfunctional requirements....7
Table 4.4 Execution environment....8
Table 6.1 Implemented grade thresholds....15
Table 7.1 Executed unit test summary....21
Table 7.2 Backend unit case outcomes....22

CHAPTER 1
INTRODUCTION

1.1 Definition of the system

The Result Processing System (RPS) is a role-based web application for organizing academic records and processing semester results. It brings department and session records, students, teachers, curriculum courses, teaching assignments, assessment marks, publication and student result viewing into one local browser-based workspace. The application is developed using FastAPI, SQLAlchemy and SQLite, with a plain HTML, CSS and JavaScript interface.

1.2 Problem statement

A result-processing workflow involves more than adding marks. Each result must belong to the correct student, course, department, academic session and semester. When separate registers or spreadsheets are maintained without a shared workflow, repeated entry, inconsistent course lists and accidental access to the wrong batch become possible. A teacher also needs a clear boundary between the courses they teach and the records they are permitted to change.

A second problem is the distinction between a student's current semester and the semester whose results are being processed. The prepared CSE 2021–2022 batch is currently in Semester 7, while the intended demonstration processes Semester 6 results. Treating these two values as the same would make past-semester result processing unnecessarily difficult.

1.3 Purpose

The purpose of RPS is to provide a small, understandable academic application in which administrators control academic setup, teachers enter marks within an assigned scope, and students view their own published results. It supports the practical sequence from registering a batch to reviewing and publishing a result, while preserving a clear separation between draft work and student-visible information.

1.4 Report organization

This report presents the project background, relevant technical concepts, feasibility, requirements, design, implementation and test evidence. Chapter 7 reports isolated unit tests of grading, input validation and frontend state. Chapter 8 discusses limitations and future work. The references identify the technical sources used throughout the book.

Objectives and scope

1.5 Objectives

- Maintain department, session, student, teacher and course records in a consistent relational database.
- Automatically assign the department’s fixed core curriculum to students of a configured academic batch.
- Assign teachers to courses for a specific session and prevent access to other sessions.
- Validate assessment marks, calculate totals and grades, and derive credit-weighted semester GPA and CGPA.
- Separate draft marks from published results and protect saved records against stale edits.
- Provide a clear user interface, a reproducible startup procedure, database backup and a practical user manual.

1.6 Scope of the implemented system

The implemented local application includes account provisioning, temporary passwords, forced first-login password change, role-based access checks, CSV imports for students and teachers, batch settings, fixed-course enrollment, teacher assignment, marks entry and result publication. The prepared curriculum contains 81 CSE courses: 53 core courses, 14 optional theory courses and 14 optional lab courses. Optional choices are made separately in Semester 8.

1.7 Project boundaries

RPS is a local academic demonstration rather than an institution-approved production service. It does not implement retake attempts, replacement of previous grades, automatic email delivery, single sign-on, printable official transcripts or a complete audit-history module. It does not claim that its configured grading policy has been formally approved by the institution. Public hosting requires additional deployment, security, backup and operational review.

1.8 Academic batch configuration

A batch is identified by its department and academic session. Its current semester determines which marks may be entered, while its processing semester selects the results being prepared. For the CSE 2021-2022 configuration, these values are 7 and 6 respectively. Keeping them separate supports the common situation in which teaching has moved forward while an earlier semester is still being processed.

CHAPTER 2
LITERATURE REVIEW

2.1 Academic result-processing approaches

Academic results can be maintained using paper registers, spreadsheet workbooks or a structured application. These approaches differ mainly in how they represent relationships, validate inputs, separate responsibilities and control publication. A spreadsheet can calculate a GPA correctly when its formulas and data are maintained carefully; the challenge addressed by RPS is the surrounding workflow, including batch identity, teaching responsibility, access control and review status.

RPS therefore models the academic entities explicitly. A student belongs to a department and session, a course belongs to a department and semester, and a teaching assignment joins a teacher, course and session. This design makes the batch boundary a condition checked by the application rather than a convention stored in a worksheet name.

2.2 Role-based access control

Role-based access control associates permissions with organizational responsibilities. NIST describes RBAC as restricting access according to user roles [3]. RPS applies this concept through three roles: administrator, teacher and student. Role checks alone are not sufficient: a teacher must also match the assigned teacher code, course and session, while a student request must match that student’s reference code.

2.3 Authentication and validation

FastAPI documents password hashing, bearer-token authentication and JWT validation as components of an authenticated application [1]. RPS uses this general approach with password hashes, expiring tokens and database-backed user checks. Input validation is performed on the server as well as in the interface. Browser controls improve usability, but the server remains responsible for rejecting invalid marks, duplicate records and unauthorized changes.

Technical foundations and comparison

2.4 Relational integrity and transactions

A relational database can express dependencies between academic records. SQLite documents foreign keys as a way to enforce that referenced records exist; the feature must be enabled for each connection [2]. RPS enables foreign-key enforcement and groups related writes within transactions. For example, a batch import is validated before insertion so that invalid rows do not leave a partially imported batch.

2.5 Optimistic concurrency

A user may review data that another user subsequently changes. SQLAlchemy's documentation discusses version counters for detecting stale records [4]. RPS implements its own revision-checked conditional SQL updates for marks and publication. It does not configure the SQLAlchemy mapper version counter. The browser submits the revision it displayed; a mismatch returns a conflict instead of silently replacing a newer result.

2.6 Scope comparison

Table 2.1 Comparison of result-processing approaches

Criterion	Separate register or workbook	Implemented RPS
Academic relationships	Maintained through conventions and links	Department, session, course and enrollment records
Teacher access	Depends on document-sharing arrangements	Role plus course and session assignment checks
Publication	Usually a separate distribution step	Explicit draft and published states
Concurrent edits	Depends on the document platform	Marks revision checked before save or publish
Deployment	Document or spreadsheet environment	Local Python application with SQLite
Institutional fit	Highly adaptable manually	Fixed project rules requiring institutional review

The comparison describes the operational scope of this project, not a measured benchmark of commercial products. RPS concentrates on a small, demonstrable workflow and avoids claiming the reporting, scalability or integration capabilities of a full university information system.

CHAPTER 3

FEASIBILITY STUDY

3.1 Operational feasibility

The three roles correspond to familiar academic responsibilities. Administrators prepare records and review results; teachers enter assessment marks; students read published results. Fixed curriculum enrollment removes repeated course assignment for every student. Separate current and processing semesters allow the prepared batch to study Semester 7 while Semester 6 results are processed.

3.2 Technical feasibility

The application runs from one local address and serves both the interface and API. SQLite avoids a separate database-server installation for the demonstration. The backend uses typed validation, relational records and transaction boundaries. No controlled load benchmark has been performed; suitability for a larger workload requires measurement.

3.3 Economic feasibility

The project relies on open-source development tools and can be demonstrated on an existing computer. The principal practical costs are development time, data preparation, testing and maintenance. Public hosting would introduce additional operational costs, including a hosting environment, backups, secure access and administration. The feasibility assessment is qualitative; institutional deployment would require a formal cost and benefit review.

3.4 Adoption and schedule feasibility

The interface uses ordinary forms and role-specific navigation. Name-and-code selectors reduce the need to memorize identifiers, and temporary-password messages support first-time users. Development proceeded through repeated requirement clarification, implementation and testing. The milestones below summarize the implemented development phases.

Table 3.1 Development milestones

Milestone	Evidence
Academic model and setup	Application models and startup commands
Role workflow	Separate admin teacher and student workspaces
Calculation and validation checks	Twelve isolated Python unit methods
Frontend state checks	Six isolated JavaScript unit methods
Documentation	Eight chapters with diagrams and interface snapshots

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Development methodology

An iterative development approach was used because operational requirements were clarified while the application was being exercised. The implementation was organized around the academic model and role-specific pages. Later changes added the complete role workflow, batch-specific teacher assignment, fixed enrollment, safer marks editing and clearer frontend screens. The unit checks in Chapter 7 isolate selected functions and state handlers from the real academic records.

4.2 Actors and responsibilities

Table 4.1 Role responsibility matrix

Operation	Admin	Teacher	Student
Manage academic records	Yes	No	No
Create or reset login accounts	Yes	No	No
Set batch semesters and assign teachers	Yes	No	No
Enter assessment marks	No	Assigned scope only	No
Review and publish marks	Yes	No	No
View semester results	All students	No student-results endpoint	Own published results
Change own password	Yes	Yes	Yes

4.3 Functional requirement groups

The system must identify the user, establish the permitted academic scope and then apply the rules for the requested operation. Administrative maintenance, curriculum enrollment, assessment, publication and result viewing form the main functional groups. The distinction between authentication, role authorization and academic ownership is maintained in both the design and tests.

4.4 Data privacy requirements

Password hashes must not be returned by account-listing endpoints. A student must not retrieve another student’s results. A teacher must not retrieve the roster or marks of a course/session they are not assigned to. Demonstration screenshots use fictional records rather than publishing the supplied students’ personal email addresses.

Functional and nonfunctional requirements

Table 4.2 Functional requirements

ID	Requirement	Acceptance condition
FR01	Authenticate users	Valid credentials return an expiring token
FR02	Provision accounts	Temporary password displayed after creation/reset
FR03	Maintain academic registries	Validated create, edit and protected delete
FR04	Import students and teachers	Preview first; invalid batch writes nothing
FR05	Configure academic batches	Processing semester does not exceed current semester
FR06	Apply fixed curriculum	Core enrollment repeats without duplicates
FR07	Assign teachers	Assignment includes teacher, course and session
FR08	Save assessment marks	Four bounded values and revision checks
FR09	Publish reviewed results	Only admin can change publication state
FR10	Calculate results	Credit-weighted GPA/CGPA with pending exclusions
FR11	Manage Semester 8 electives	At most one optional theory and one lab
FR12	Back up local data	SQLite backup saved without changing records

Table 4.3 Nonfunctional requirements

Quality	Implemented measure
Security	Role/ownership checks, password hashes and token expiration
Integrity	Foreign keys, unique constraints and transaction boundaries
Usability	Role-specific navigation, search and clear confirmation controls
Reliability	Conflicts rejected instead of overwriting newer marks
Maintainability	Separated models, database setup, service logic and tests
Portability	Local browser interface and documented Python startup
Recoverability	Backup command and manual restoration procedure

Business rules and execution environment

4.5 Core business rules

- A configured batch is identified by department and academic session. Current semester and processing semester are separate values.
- All fixed core courses are enrolled automatically, including future-semester core courses. A future enrollment does not create a result.
- Teacher permissions are restricted by teacher code, course code and session. Future-semester marks remain locked until the batch reaches that semester.
- Assessment maxima are assignment 10, class test 20, midterm 40 and final 80. All four values are required and allow at most two decimal places.
- Students see published results only. Published marks must be unpublished by an administrator before teacher revision.
- Saved results protect relevant academic data: credits cannot change after marks exist, and course type cannot change after enrollment.

4.6 Software and hardware environment

Table 4.4 Execution environment

Component	Project configuration
Backend	Python, FastAPI 0.141.1 and Uvicorn 0.53.0
Persistence	SQLAlchemy 2.1.4 and SQLite
Validation and security	Pydantic 2.13.5, PyJWT 2.15.1 and pwdlib 0.3.1
Frontend	HTML, CSS and JavaScript without a frontend build step
Local address	http://127.0.0.1:8001
Practical equipment	A computer with Python, a modern browser and available disk space

These are the pinned project dependencies, not a claim that they are the latest releases. The supplied development environment uses a Python virtual environment. Hardware figures are not presented as benchmarked minimum requirements; the application should be checked on the actual demonstration computer before submission.

CHAPTER 5

SYSTEM DESIGN

5.1 Architectural design

RPS uses a browser interface, a FastAPI application layer and a relational persistence layer. The frontend calls the API from the same origin and does not open the SQLite database directly. Authentication and academic authorization are checked before business logic executes. SQLAlchemy sessions provide the boundary through which validated changes reach the database.

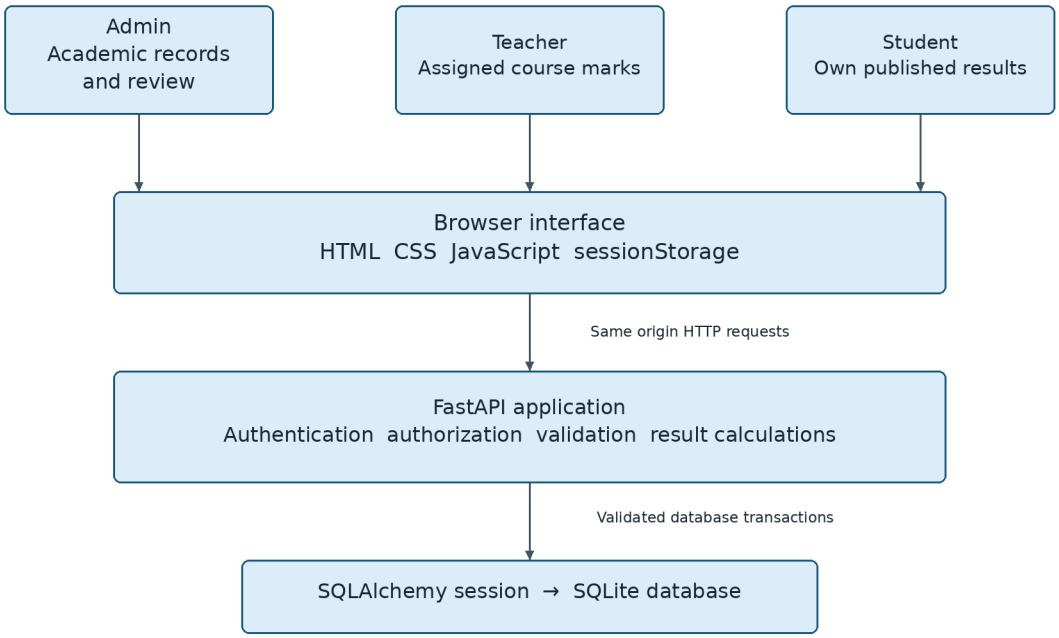


Figure 5.1 Layered architecture of the Result Processing System

5.2 Input and output design

Input forms use labels, bounded numeric fields, selection lists and human-readable errors. Names and public codes appear together when choosing a teacher or account owner. Outputs include record lists, academic-batch summaries, draft marks, publication status and student semester results. Generated totals and grade points are calculated by the server rather than trusted from browser input.

Use case design

5.3 Use case diagram

The administrator establishes the academic workspace and controls publication. Teachers act within assigned offerings, and students read their own results. Password changes are available to every authenticated role; account creation remains an administrative operation.

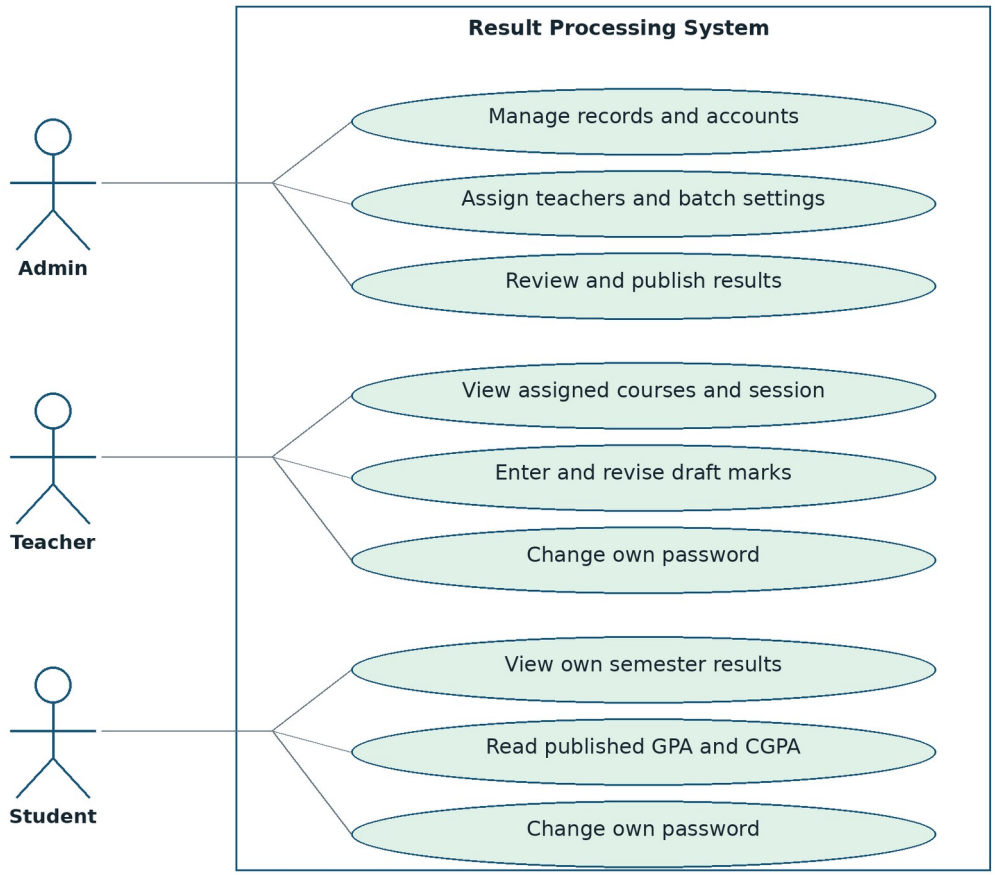


Figure 5.2 Role-specific use cases

A successful login does not grant every use case. Every protected operation still evaluates the required role and, where relevant, the session and owner of the requested record.

Data flow design

5.4 Data flow diagram

Academic setup, marks entry and published-result viewing exchange data through the application. Administrative publication changes the state of an existing marks record; it does not create a separate copy of a result. A student query therefore filters the same underlying records by publication status.

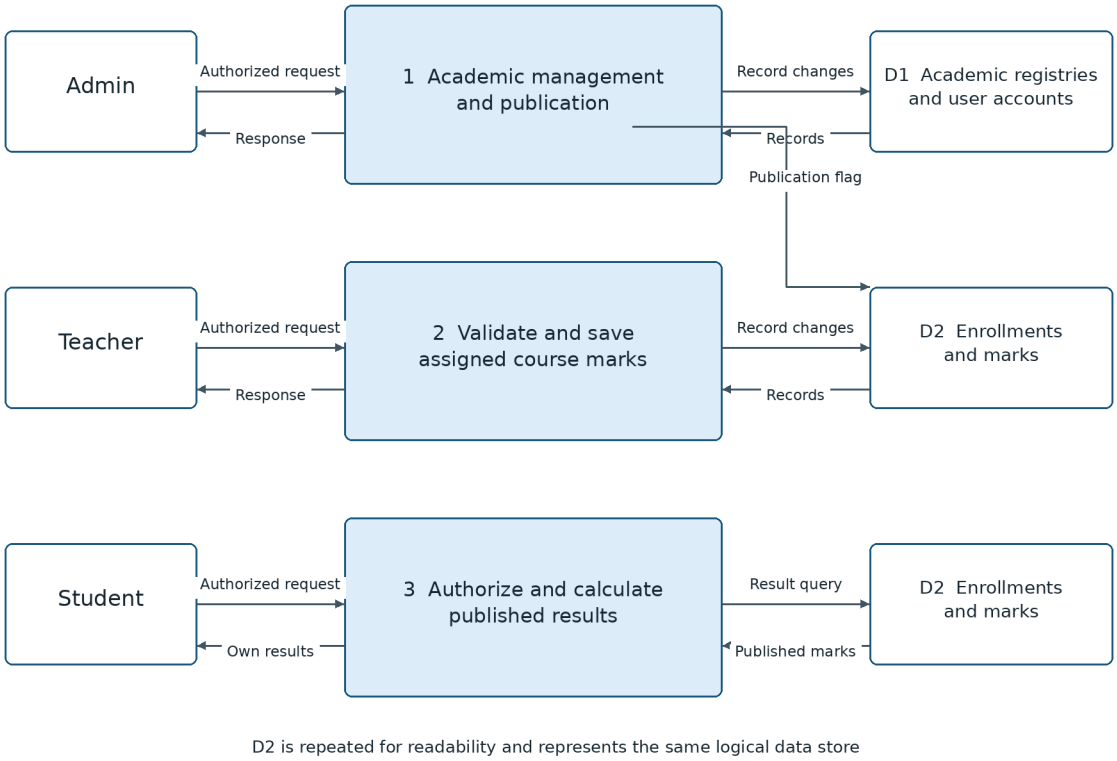


Figure 5.3 Main academic data flows and logical data stores

5.5 Transaction boundaries

CSV import and fixed enrollment are grouped with their associated record changes. Preview operations return validation results without saving. Optional-course removal is blocked when saved marks exist. Deletion checks use conditional database operations to avoid removing a result that was saved while an administrative request was in progress.

Database design

5.6 Entity relationships

The schema contains ten tables. Departments and academic sessions identify batches; students and teachers are registered separately from login accounts. Courses form the static curriculum. Enrollments connect students to courses, and marks connect to a unique enrollment. The diagram identifies the principal keys and relationships; account ownership uses an application-level reference.

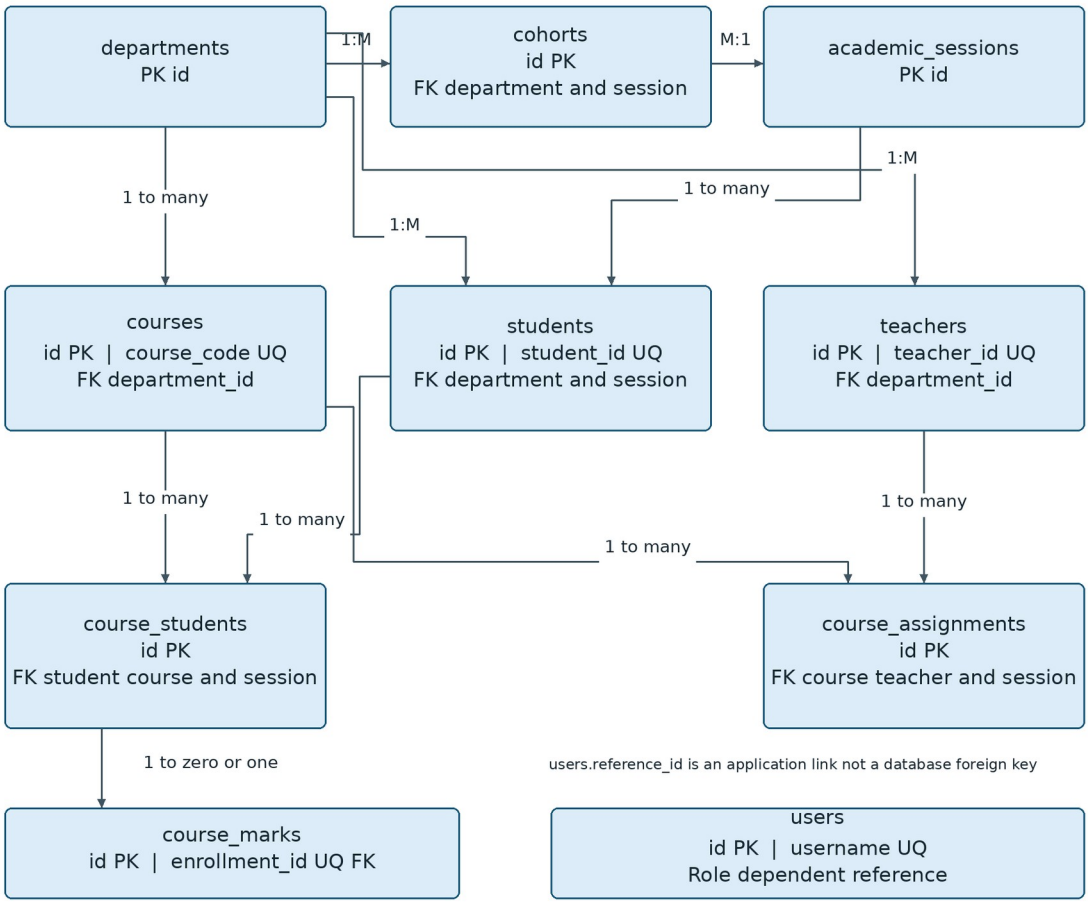


Figure 5.4 Database entities and principal relationships

The user reference code is validated by application logic and is not a database foreign key because it can identify either a teacher or a student. Enrollment stores session and semester explicitly; the API keeps these values aligned with the student and course. A course may have several teachers for a session.

Sequence and state design

5.7 Result processing sequence

Marks entry, administrative review and student viewing are separate actions. A revision number connects what a user reviewed with what the database is asked to modify. Publication increments the revision, so an older teacher screen cannot silently save over a publication decision.

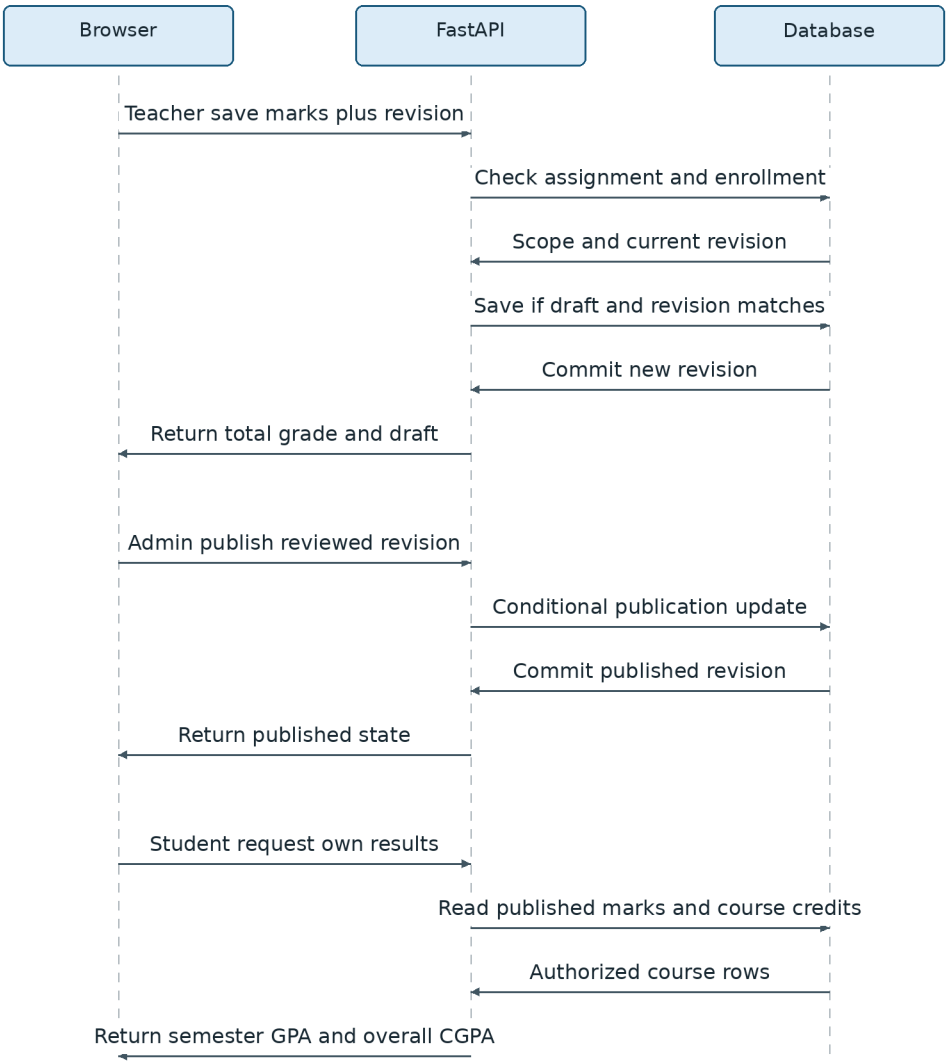


Figure 5.5 Save review publish and view sequence

The operational state changes are: no saved marks → draft → published. Unpublishing returns a record to draft. Each successful save or publication change advances the revision; a stale request receives a conflict and must reload.

CHAPTER 6

IMPLEMENTATION AND RESULTS

6.1 Application structure

```
RPS/
  Backend/
    app/main.py              API and business rules
    app/models/              Academic and account models
    app/db/                  Connection and schema upgrades
    app/services/curriculum.py  Fixed enrollment synchronization
    app/routes/imports.py    CSV validation and import
    tests/test_system.py     HTTP integration tests
    manage.py                Initialization admin setup backup
    requirements.txt         Pinned backend dependencies
  Frontend/
    index.html  app.js  style.css
    tests/state.test.cjs      Frontend state tests
  start.sh                   Local startup helper
```

6.2 Database initialization and migration

The local database is stored separately at Backend/rps.sqlite3. The application deliberately does not reuse the original DATABASE_URL setting. Startup initializes the schema, applies supported local upgrades and synchronizes configured core enrollments. A legacy assignment without a session mapping is not guessed: startup stops with an explanatory error rather than assigning it to every session.

6.3 Account lifecycle

The first administrator is created through an interactive local command. Other accounts are created from the Accounts page. A generated temporary password is shown immediately after creation or reset and must be shared privately. First-time users change that password before accessing academic pages. Password changes and resets invalidate existing tokens through a token-version check.

Grading and result calculation

6.4 Assessment total and grade mapping

For each course, total marks equal assignment plus class test plus midterm plus final. The total is out of 150. Grade thresholds are percentages of that total; for example, 120 out of 150 is 80 percent and maps to A+. All courses currently use the same assessment structure, including lab and project courses. An alternative institutional structure requires a deliberate policy change before real use.

Table 6.1 Implemented grade thresholds

Minimum percentage	Minimum total out of 150	Grade	GP
80	120.00	A+	4.00
75	112.50	A	3.75
70	105.00	A−	3.50
65	97.50	B+	3.25
60	90.00	B	3.00
55	82.50	B−	2.75
50	75.00	C+	2.50
45	67.50	C	2.25
40	60.00	D	2.00
Below 40	Below 60.00	F	0.00

6.5 GPA and CGPA

Semester GPA = sum of course credit × grade point for eligible courses, divided by their total credits. CGPA applies the same calculation across all semesters, even when one semester is selected for viewing. Student calculations use published marks only; administrative previews include saved draft marks. Failed eligible courses count with zero grade points. Pending courses are excluded and the semester is marked Incomplete.

Decimal arithmetic avoids avoidable floating-point artifacts. A verified example with two three-credit courses having grade points 4.00 and 3.00 gives (3 × 4.00 + 3 × 3.00) ÷ 6 = 3.50. No retake or grade-replacement rule is implemented.

Login interface

6.6 Login interface and first access

The login page provides one entry point for all roles. The server determines the correct workspace after authenticating the user. The page identifies the academic portal, provides clearly labelled inputs and displays the developer credit. The screenshot below shows the shared sign-in interface.

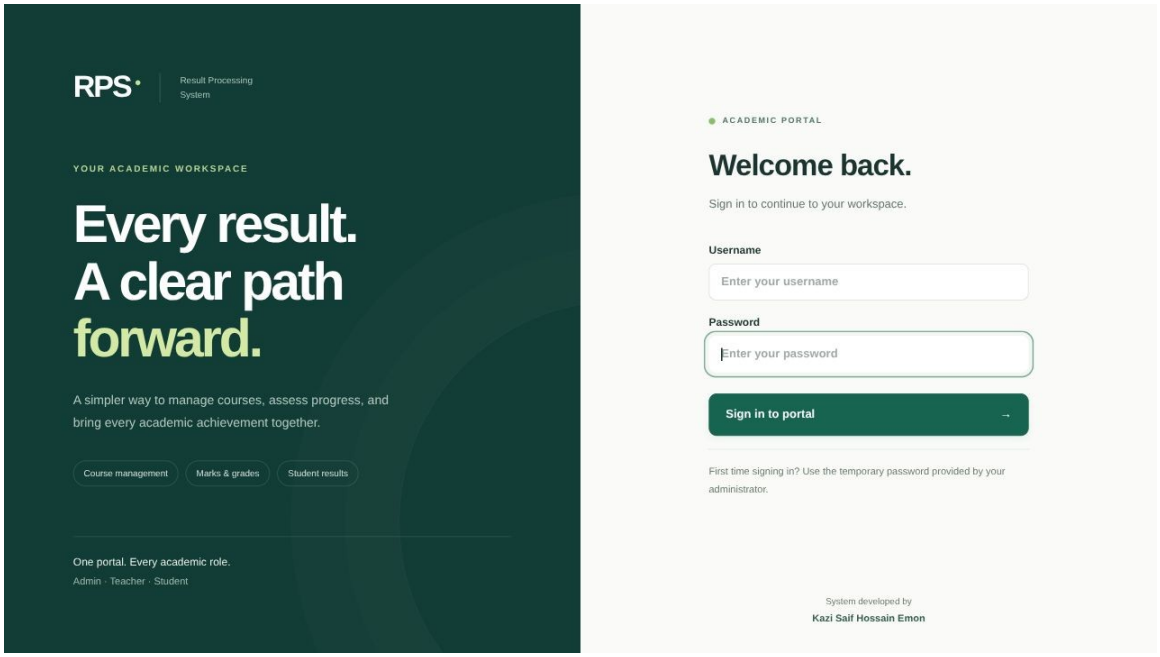


Figure 6.1 Redesigned login interface

6.7 Interface conventions

The interface uses dark green navigation, restrained cards, consistent controls and readable feedback. Role-specific navigation keeps unrelated operations out of each workspace. The footer displays “System developed by Kazi Saif Hossain Emon”. A responsive layout rearranges the navigation and cards for narrower screens; wide record tables remain horizontally scrollable when necessary.

For account recovery, administrators generate a new temporary password rather than retrieving the old password. Plaintext passwords are not stored in the account table, and the interface does not display existing password hashes.

Administrative dashboard

6.8 Dashboard and batch summary

The dashboard summarizes students, teachers, curriculum courses and course registrations. Batch progress displays the current semester separately from the result-processing semester. Interface illustrations in this chapter use fictional review records and sample grades, not official student results.

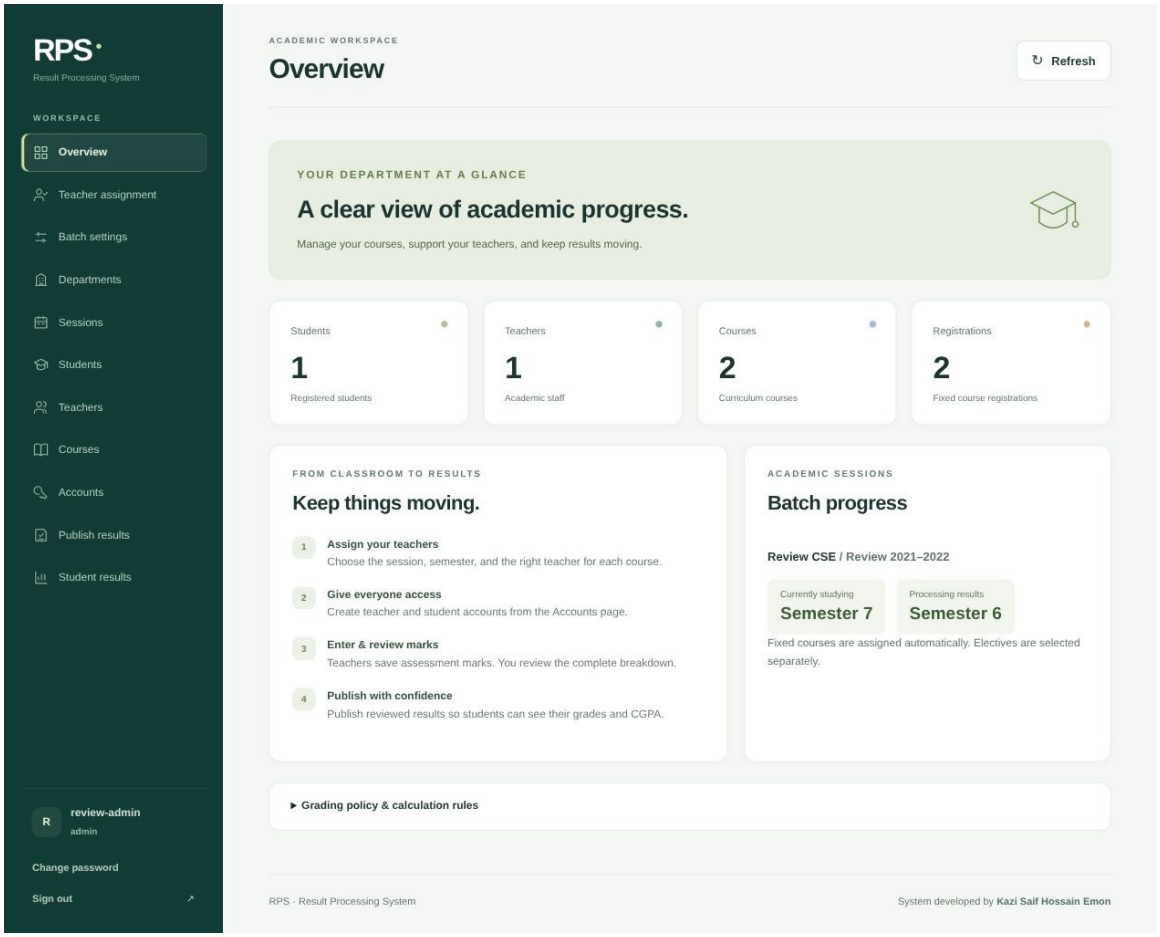


Figure 6.2 Administrative dashboard with batch progress

Teacher assignment and Batch settings are separate, visible navigation items. The removed Assignments and Enrollments menu pages are not required for the normal fixed-curriculum workflow.

Teacher and administrator workspaces

6.9 Teacher assignment and marks entry

The administrator selects a department, session and semester, then chooses a teacher beside a course card. Both the teacher’s name and code are visible. The teacher’s own workspace lists only assigned offerings, distinguishes sessions and provides searchable student cards with all four assessment fields and a Save marks button.

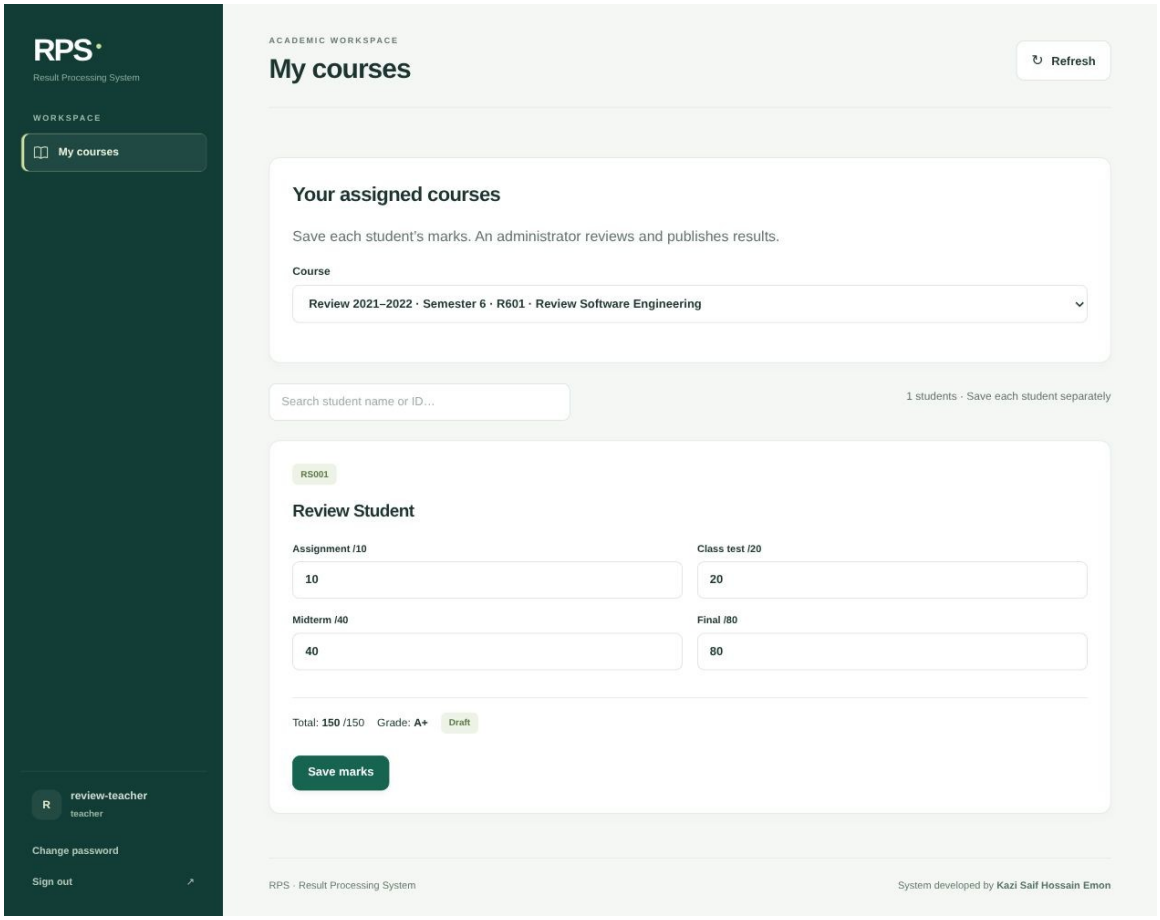


Figure 6.3 Teacher marks entry in an assigned session

Administrative result review

6.10 Administrative review

The Publish results page shows each saved result with its four assessment components, total, grade and grade point. Department, session, semester and text search narrow the list. An in-page confirmation is required before publication. If the record changed after review, publication is rejected and the administrator must reload the latest marks.

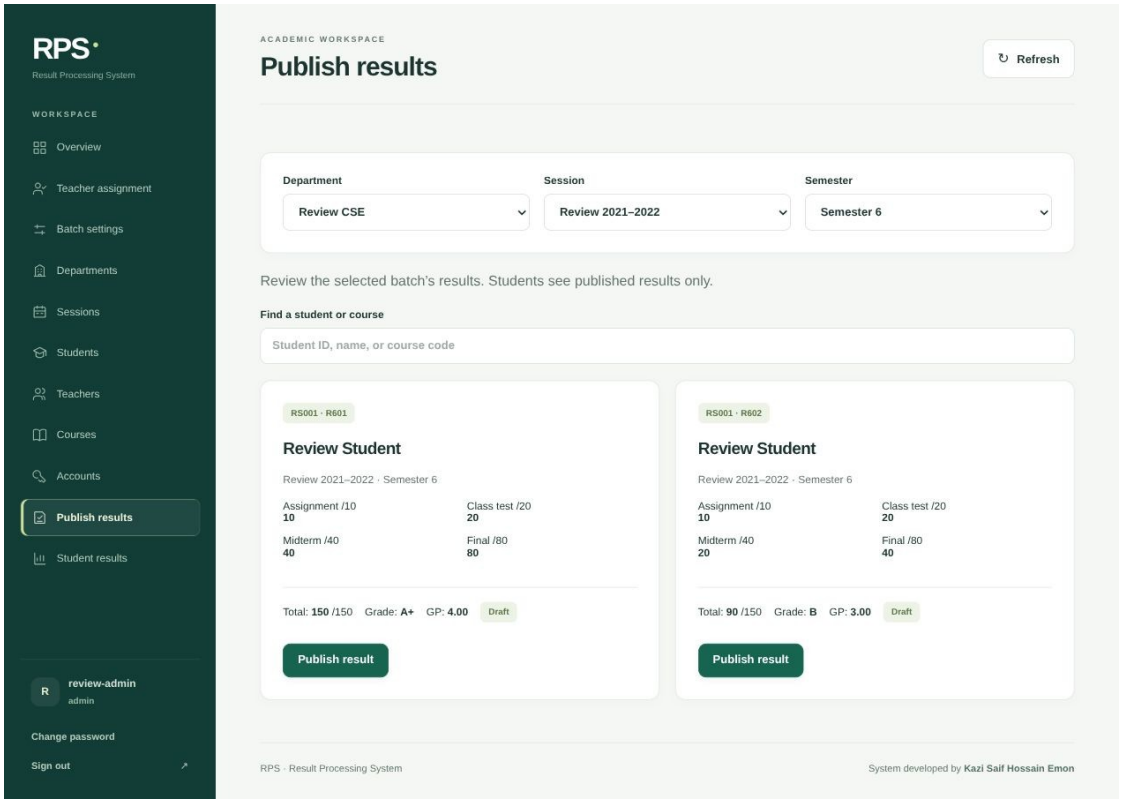


Figure 6.4 Component-level result review before publication

Student results and implementation outcome

6.11 Student result viewing

A student’s result page defaults to that student’s batch processing semester. It shows the student identity, current and processing semesters, overall CGPA and semester courses. Unpublished components are not returned to the student. The same backend ownership check prevents a modified URL from revealing a different student’s results.

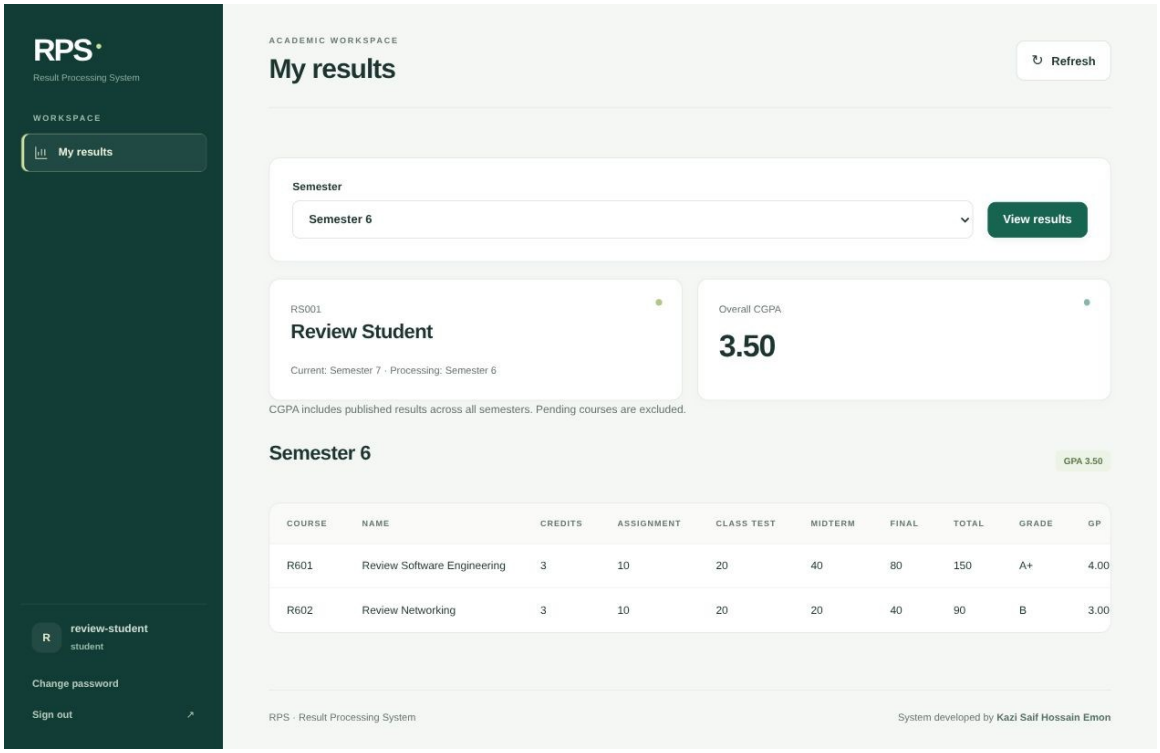


Figure 6.5 Student result view with verified GPA and CGPA

6.12 Implemented outcome

The captured demonstration illustrates two fictional course results with grade points 4.00 and 3.00 and an equal-credit GPA and CGPA of 3.50. The prepared real-data workspace is separate from that review fixture. Successful sample operation is evidence of the tested workflow, not an institutional acceptance certificate or a high-concurrency performance benchmark.

CHAPTER 7
TESTING

7.1 Unit testing approach

Unit testing examines a small function, validation model or state handler in isolation. Python unittest supplies assertions, test cases and reporting [7]. This chapter reports only unit tests. The Python checks call the implemented grading function and Pydantic marks model directly. JavaScript checks execute frontend handlers inside a virtual machine with mocked document, storage and fetch objects. No live browser, HTTP server or academic database is used by these tests.

7.2 Test environment and isolation

The Python unit runner sets an in-memory database configuration and an isolated signing key before importing the application. It does not start the application lifespan or connect to the project database. The frontend fixture supplies small stand-ins for page elements and API responses. Each test receives a fresh fixture, so unsaved marks and authentication state do not leak between cases.

7.3 Selection of cases

Grading cases target every configured threshold, the values immediately below pass and A+, zero and maximum marks. Validation cases cover permitted precision, omitted components, component limits, nonfinite input and invalid revisions. Frontend cases focus on confirmation outcomes, preservation of unsaved input, forced logout and stale asynchronous responses. The suite therefore checks both accepted and rejected input rather than only the normal path.

Table 7.1 Executed unit test summary

Unit group	Test methods	Passed	Failed
Grading and pending result functions	5	5	0
Marks input model	7	7	0
Frontend isolated state handlers	6	6	0
Total	18	18	0

Unit test cases

7.4 Backend unit cases

Table 7.2 Backend unit case outcomes

ID	Unit and input	Expected outcome
U01	Every grade threshold from 120 to 60	Correct grade and grade point
U02	Total 59.99	F and GP 0
U03	Totals 0 and 150	F and A+ respectively
U04	Total 119.99	A and GP 3.75
U05	marks_info with no saved marks	Pending values and unpublished
U06	Four components at maxima	Validation succeeds
U07	Final marks 65.25	Validation succeeds
U08	Final marks 65.251	Validation error
U09	Negative or excessive component	Validation error for each field
U10	Final component omitted	Validation error
U11	NaN or infinity	Validation error
U12	Revision zero	Validation error

7.5 Frontend unit cases

All six frontend tests passed: confirmation and cancellation resolve correctly; cancellation preserves unsaved marks while confirmation discards them; forced logout clears open dialogs and dirty state; an old unauthorized response does not clear a newer login; malformed server responses become readable errors; and a CSV preview dialog can remain open before the commit step.

Each row summarizes a test method. Some methods contain several subcases, so the 18-method total is not a claim that only 18 input values were checked. The complete runnable Python evidence is retained with the book output, and the frontend unit suite is part of the project source [6].

Unit testing snapshot

7.6 Observed execution output

The following snapshot reproduces the captured unit-runner output from 3 November 2026. It records twelve successful Python test methods and six successful JavaScript test methods. It is execution evidence for these isolated units, rather than evidence of whole-system correctness.

```
test_all_grade_thresholds ... ok
test_just_below_a_plus ... ok
test_just_below_pass_fails ... ok
test_missing_marks_remain_pending ... ok
test_zero_and_maximum ... ok
test_accepts_two_decimal_places ... ok
test_component_limits ... ok
test_missing_component_rejected ... ok
test_nonfinite_values_rejected ... ok
test_rejects_three_decimal_places ... ok
test_revision_must_be_positive ... ok
test_valid_full_marks ... ok

----------------------------------------------------------------------
Ran 12 tests in 0.001s

OK

Frontend state unit tests
tests 6    pass 6    fail 0
Source runner: node Frontend/tests/state.test.cjs
```

Figure 7.1 Captured Python and frontend unit test output

7.7 Interpretation and limits

All selected unit methods passed. These checks support the tested grading boundaries, validation constraints and state transitions. They do not establish load capacity, production security, institutional acceptance or complete workflow correctness. Credit-weighted GPA is explained mathematically in Chapter 6; it is not reported here as a separately isolated function test because the current implementation computes it within the result endpoint.

CHAPTER 8
CONCLUSIONS AND FUTURE WORK

8.1 Conclusion

RPS provides a working local path from academic setup to student result viewing. Its main contribution is the explicit connection between department, session, curriculum, teaching responsibility and publication state. Fixed core enrollment removes repetitive student-course assignment, while separate current and processing semesters support delayed result processing for a batch that has already progressed.

The implemented checks address practical failure modes: invalid marks, access to another session, stale edits, publication of an outdated review, accidental changes to credits after marks exist, and deletion of enrollments that have saved results. The user interface presents these rules through ordinary forms, selectors, student cards and confirmation controls.

8.2 Limitations

- Assessment weights are fixed at 10, 20, 40 and 80 for all course types.
- Retakes, repeated attempts and grade replacement are not modeled.
- Account credentials must be shared manually; there is no automated email or SSO.
- Official transcript generation and downloadable result sheets are not implemented.
- The local application has process-local login limits, no complete audit-history module and no production hosting configuration.

8.3 Future development

Useful extensions include institution-approved assessment policies per course type, retake handling, a verifiable result-change audit trail, student-specific elective planning, printable transcripts, notification delivery and a reviewed production deployment. These should be added only after requirements are confirmed and the existing grade calculations and access controls remain covered by tests.

8.4 Completion status

The local software workflow is implemented, and the isolated unit checks reported in Chapter 7 pass. Final academic demonstration still requires administrator-selected teaching assignments, teacher/student accounts and approved demonstration marks. Production readiness requires separate institutional and operational acceptance.

References

[1] FastAPI, “OAuth2 with Password and hashing Bearer with JWT tokens,” FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/. [Accessed: Nov. 3, 2026].

[2] SQLite, “SQLite Foreign Key Support,” SQLite Documentation. [Online]. Available: https://www.sqlite.org/foreignkeys.html. [Accessed: Nov. 3, 2026].

[3] National Institute of Standards and Technology, “Role based access control,” CSRC Glossary. [Online]. Available: https://csrc.nist.gov/glossary/term/role_based_access_control. [Accessed: Nov. 3, 2026].

[4] SQLAlchemy, “Configuring a Version Counter,” SQLAlchemy 2.1 Documentation. [Online]. Available: https://docs.sqlalchemy.org/en/21/orm/versioning.html. [Accessed: Nov. 3, 2026].

[5] K. S. H. Emon, “Result Processing System,” application source code, version 2.1.0, Institute of Science & Technology, 2026. Local project repository, reviewed Nov. 3, 2026.

[6] K. S. H. Emon, “RPS project documentation and unit test evidence,” README.md, Frontend/tests/state.test.cjs and test_rps_units.py, 2026.

[7] Python Software Foundation, “unittest — Unit testing framework,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/unittest.html. [Accessed: Nov. 3, 2026].